

Архитектурный документ. Ядро PySATL

Михаил Михайлов, Леонид Елкин

5 января 2026 г.

Оглавление

1	Введение	2
1.1	Назначение системы	2
1.2	Область применимости	2
1.3	Общие сведения о системе	2
2	Глоссарий	3
3	Заинтересованные лица	10
3.1	Разработчики ядра PySATL	10
3.2	Разработчики других библиотек в PySATL	10
3.3	Руководители проекта PySATL	11
3.4	Инженеры-исследователи	12
4	Ключевые требования, определяющие архитектуру	14
4.1	Сценарии использования системы	14
4.2	Технические ограничения и качественные требования	14
4.2.1	Технические ограничения	14
4.2.2	Качественные характеристики системы	14
4.3	Ключевые функциональные требования	15
4.3.1	Требования к числовым и функциональным характеристикам	15
4.3.2	Требования к семействам распределений	16
4.3.3	Требования к операциям над распределениями	16
4.3.4	Требования к генерации выборок	16
5	Избранные архитектурные точки зрения	17
5.1	Контекст	17
5.2	Композиция	17
5.2.1	Состав компонент	17
5.2.2	Роли компонент	18
5.2.3	Потоки данных и точки соприкосновения	18
5.3	Логическая структура	19
5.3.1	Общий обзор	19
5.3.2	Модуль <code>distributions</code>	20
5.3.3	Модуль <code>families</code>	24
5.4	Взаимодействия и потоки выполнения	26
5.4.1	Поток вычисления характеристики	26
5.4.2	Опции вычислений (<code>**options</code>)	26
5.4.3	Поток генерации выборки	26
5.4.4	Точки расширения	26

Введение

1.1. Назначение системы

Вычислительное ядро проекта PySATL предназначено для представления и обработки вероятностных распределений в программной форме. Ядро предоставляет средства для задания распределений и их семейств, выполнения операций над ними и построения более сложных структур путём функциональных и алгебраических преобразований.

Система поддерживает задание как конкретных распределений с определёнными параметрами, так и абстрактных семейств, из которых могут быть получены конкретные экземпляры. Кроме того, предусмотрена возможность определения пользовательских распределений и преобразований, расширяющих базовые возможности.

Ядро служит универсальной основой для статистических и вероятностных вычислений в рамках проекта PySATL и может использоваться другими подсистемами при построении моделей.

1.2. Область применимости

Вычислительное ядро `core` используется во всех подсистемах проекта PySATL, где требуется работа с распределениями вероятностей. Оно предназначено как для непосредственного вычисления характеристик распределений (например, плотности, функции распределения, квантилей), так и для построения и трансформации более сложных моделей на их основе. Оно может быть использовано:

- при определении конкретных распределений, используемых в анализе данных;
- для задания пользовательских распределений, комбинации распределений и создания новых семейств;
- при трансформации распределений через функциональные отображения;
- в задачах символьной или численной обработки распределений.

Вне проекта PySATL ядро может быть применимо в любых системах, где необходима гибкая и расширяемая работа с вероятностными распределениями, особенно в контексте численных симуляций, статистического моделирования и прикладного машинного обучения.

1.3. Общие сведения о системе

Ядро представляет собой модульную систему, реализованную на языке Python, предназначенную для работы с вероятностными распределениями, их семействами и преобразованиями. Архитектура системы построена на разделении функциональности между независимыми компонентами, каждый из которых отвечает за определённый класс задач.

Компоненты взаимодействуют друг с другом через чётко определённые интерфейсы. Такая организация позволяет изолировать ответственность отдельных частей системы, обеспечивать гибкость при расширении функциональности и облегчать сопровождение кода.

Некоторые интерфейсы, предоставляемые одним модулем, используются другими модулями для построения более сложных вычислений или абстракций. Это позволяет комбинировать базовые элементы в составные структуры и формировать цепочки преобразований.

Проект спроектирован таким образом, чтобы допускать расширение без модификации существующих компонентов, в соответствии с принципами модульности и открытости/закрытости. Это обеспечивает стабильную основу для развития ядра и его интеграции с другими частями проекта PySATL.

Глоссарий

Основным понятием в статистике и стохастическом моделировании является *распределения случайной величины* или, более общо, *распределения случайного объекта* (далее, под случайной величиной понимается любой случайный объект, реализации которого не обязательно суть вещественные числа) [18]. Ниже изложены основные теоретические сведения касающиеся случайных величин, а также задач в которых они возникают, в соответствии с монографиями [37] и [39].

Случайные величины и способы их задания

Для случайной величины ξ , принимающей значения в некотором пространстве $\mathcal{X}$, её распределением называется (см. [39]) вероятностная мера $\mathbb{P}_\xi(\cdot)$ на $\mathcal{X}$, такая что $\mathbb{P}_\xi(A)$ есть вероятность того что реализация ξ попадет в множество $A \subset \mathcal{X}$ ¹. Как правило (см. [37]), выделяют следующие виды случайных величин

- Дискретные случайные величины. В этом случае $\mathcal{X}$ представляет собой некоторое дискретное (конечное или счетное) множество. Например число выпадений монеты орлом при нескольких бросках (биномиальное распределение); уровень образования у случайно выбранного человека (категориальное распределение); случайная величина которая принимает одно значение (вырожденное распределение);
- Одномерные непрерывные случайные величины². В этом случае $\mathcal{X} = \mathbb{R}$ или $\mathcal{X} \subset \mathbb{R}$ ненулевой меры. Такие величины используются для описания случайных времен, расстояний и т.д. Согласно [18] наиболее важными представителями являются: равномерное распределение $\mathcal{U}(a; b)$, нормальное распределение $\mathcal{N}(\mu, \sigma^2)$ и экспоненциальное распределение $\text{Exp}(\lambda)$;
- Многомерные непрерывные случайные величины. В этом случае $\mathcal{X} \subseteq \mathbb{R}^d$, ненулевой меры. Во много многомерные случайные величины являются аналогами одномерных непрерывных случайных величин, однако решение стандартных задач, таких как моделирование или вычисление числовых характеристик затруднено из-за проклятия размерности [7].

Отдельное направление статистики работает с данными о направлении (англ. directional data), в связи с этим часто можно также отдельно выделить следующую категорию случайных величин.

- Случайные геометрические примитивы (англ. geometrical random primitives). Примерами таких случайных величин служат случайные углы или случайные матрицы симметрий. Согласно [29], геометрической случайной величиной называется случайная величина принимающая значения на замкнутой и ограниченной поверхности в евклидовом пространстве (более общо—компактном Римановом многообразии).

С точки зрения ПО, работа с распределением, как с вероятностной мерой, является неудобной, так как компьютер не может работать с произвольными множествами. Однако, как правило, с распределением можно связать некоторую функцию, которая полностью определяет распределение. Так, чтобы идентифицировать распределение дискретной случайной величины, достаточно знать *функцию вероятности*, определяемую равенством (pmf).

$$f_\xi(x) = \mathbb{P}_\xi(\{x\}), \text{ т.е. вероятность того что } \xi = x, x \in \mathcal{X} \quad (\text{pmf})$$

Если на пространстве возможных значений $\mathcal{X}$ задана некоторая мера μ , плотностью распределения $\mathbb{P}_\xi$ относительно μ называется³ такая функция $f_\xi(x): \mathcal{X} \rightarrow \mathbb{R}$, что выполнено тождество (pdf):

$$\mathbb{P}_\xi(A) = \int_A f_\xi(x) d\mu \quad (\text{pdf})$$

В случае если μ это считающая мера, плотность f_ξ определяется равенством (pmf), в случае если $\mu = m_{\text{Leb}}$, говорят просто о плотности непрерывной случайной величины/случайного вектора.

¹Строго говоря, $\mathcal{X}$ должно быть снабжено некоторой σ -алгеброй $\mathcal{F}$, и $\mathbb{P}_\xi$ должна быть определена только для $A \in \mathcal{F}$. Иначе говоря, тройка $(\mathcal{X}, \mathcal{F}, \mathbb{P}_\xi)$ должна образовывать вероятностное пространство.

²Здесь и далее под непрерывными случайными величинами подразумеваются абсолютно-непрерывные случайные величины, т.е. распределение которых имеет плотность относительно меры Лебега

³Условия существования плотности описываются теоремой Радона-Никодима, см. например [21]

Несмотря на то что плотность распределения полностью его характеризует, для того чтобы вычислять вероятности $\mathbb{P}_\xi(A)$ необходимо производить интегрирование (или суммирование), поэтому для некоторых задач представление распределения в виде плотности является неудобным. В частности, если ξ — случайная величина (дискретная или непрерывная) принимающие значения из $\mathbb{R}^d$, довольно часто приходится смотреть на вероятность попадания в некоторую ячейку $\langle \mathbf{a}; \mathbf{b} \rangle$. Под ячейкой подразумевается множество:

$$\langle \mathbf{a}; \mathbf{b} \rangle = \left\{ \begin{pmatrix} c_1 \\ \vdots \\ c_d \end{pmatrix} \in \mathbb{R}^d \mid a_1 < c_1 \leq b_1, \dots, a_d < c_d \leq b_d \right\}$$

Для доступа к таким вероятностям эффективнее работать с *функцией распределения случайной величины*, определяемой равенством (cdf).

$$F_\xi(\mathbf{x}) = \mathbb{P}_\xi(\langle -\infty; \mathbf{x} \rangle) \quad (\text{cdf})$$

В этом случае $\mathbb{P}_\xi(\langle a; b \rangle)$ выражается через значения $F_\xi(\cdot)$ с помощью формулы включения-исключения [37].

С понятием функции распределения тесно связано понятие квантильной функции. Для случайной величины ξ со значениями из $\mathbb{R}$, её квантильная функция определяется равенством (ppf) (подробно о различных определениях см. в обзоре [20]).

$$\omega_\xi(p) = \inf_u \{F_\xi(u) \geq p\} \quad (\text{ppf})$$

Такое определение гарантирует, что случайная величина $\omega_\xi(U)$, $U \sim \mathcal{U}[0; 1]$ имеет такое же распределение как и сама величина ξ . В случае когда $F_\xi(\cdot)$ строго возрастает на всей области определения, квантильная функция является обратной функцией $\omega_\xi(\cdot) = F_\xi^{-1}(\cdot)$. Отдельно следует отметить что существуют обобщения понятия квантильной функции на случай случайных величин со значениями из $\mathbb{R}^d$ [12], однако для их вычисления необходимо решать уравнения в частных производных [10].

Существуют и другие функциональные характеристики распределения, многие из которых приходят из анализа выживаемости [17]. *Функцией выживаемости* называется функция определяемая равенством (sdf).

$$S_\xi(\mathbf{x}) = 1 - F_\xi(\mathbf{x}) \quad (\text{sdf})$$

Для случайных величин со значениями из $\mathbb{R}$, функцией интенсивности отказов и кумулятивной функцией интенсивности отказов называются функции определяемые равенствами (hrdf) и (chdf) соответственно.

$$h_\xi(x) = -\frac{S'_\xi(x)}{S_\xi(x)}; \quad (\text{hrdf})$$

$$H_\xi(x) = -\ln(S_\xi(x)); \quad (\text{chdf})$$

Однако, некоторые распределения, например α -устойчивые распределения [37], не допускают явного задания с помощью плотности или функции распределения, однако допускают задания с помощью так называемых *интегральных преобразований*. Такие распределения все чаще возникают в современных моделях стохастического анализа, (см. например [4]). В случае случайной величины ξ со значениями из $\mathbb{R}$, её

- Характеристической функцией ξ называется преобразование Фурье, определяемое равенством (cf).

$$\varphi_\xi(u) = \int_{\mathbb{R}} \exp(itu) \mathbb{P}_\xi(dt), \quad u \in \mathbb{R} \quad (\text{cf})$$

- Момент-производящей функцией называется преобразование определяемое равенством (mgf).

$$M_\xi(u) = \int_{\mathbb{R}} \exp(tu) \mathbb{P}_\xi(dt), \quad u \in \mathbb{R} \quad (\text{mgf})$$

Характеристическая функция всегда существует и полностью определяет распределение случайной величины [39]. В свою очередь, момент-производящая функция существует не всегда, но в тех случаях когда существует, также однозначно характеризует распределение. В случае когда ξ принимает только неотрицательные значения, определены также преобразование Лапласа и преобразование Меллина, задаваемые равенствами (lt) и (mt) соответственно.

$$\mathcal{L}_\xi(u) = \int_{\mathbb{R}_+} \exp(-tu) \mathbb{P}_\xi(dt), \quad u \in \mathbb{R}_+ \quad (\text{lt})$$

$$\mathcal{M}_\xi(u) = \int_{\mathbb{R}_+} t^u \mathbb{P}_\xi(dt), \quad u \in \mathbb{R}_+ \quad (\text{mt})$$

Эти преобразования также однозначно характеризуют распределение ξ [37], [11]. В случае если ξ многомерная случайная величина, также определяется характеристическая функция (см. [39]), преобразования (mgf), (lt), (mt) в некоторых ситуациях допускают обобщение на многомерный случай, см. например [2].

На рис. 2.1 схематично изображены основные способы задания непрерывных вероятностных распределений, и связь между ними. В связи с тем что в теории многомерных квантилей нет результатов напрямую выражающих квантильные функции через плотности или интегральные преобразования, переходы которые имеют место быть только в одномерном случае, изображены пунктирными стрелками.



Рис. 2.1: Способы задания непрерывных распределений

Замечание. Плотность и функция распределения непрерывной случайной величины со значениями в $\mathbb{R}^d$ связаны соотношениями

$$f_{\xi}(\mathbf{x}) = \frac{\partial F_{\xi}}{\partial x_1 \dots \partial x_d}(\mathbf{x}) \quad F_{\xi}(\mathbf{x}) = \int_{(-\infty; \mathbf{x})} f_{\xi}(\mathbf{t}) d\mathbf{t} \quad (2.1)$$

Формулы обращения для интегральных преобразований представлены в [39], [11]. Отдельно стоит отметить, что в работе [34] показано как можно вычислять квантильную функцию по плотности распределения и наоборот, не прибегая к вычислению функции распределения. Этот подход может оказаться полезным при работе с достаточно сложными плотностями.

Семейства вероятностных распределений

В задачах статистики, как правило, оперируют не с одним каким-то конкретным распределением, а с набором распределений, из которого надо выбрать наиболее подходящее, или проверить какую-то гипотезу. Более строго, *параметрическим семейством распределений* называется некоторое множество $\{\mathbb{P}_{\theta}\}_{\theta \in \Theta}$ распределений, зависящих от скалярного или векторного параметра θ , Θ — множество возможных значений параметра [38].

Для любого распределения $\mathbb{P}_{\xi}$ случайной величины ξ определено семейство локации и масштаба, т.е. семейство распределений всех аффинных преобразований величины ξ :

$$\text{loc} + \text{scale} \cdot \xi \sim \mathbb{P}_{(\text{loc}, \text{scale})}^{\xi}; \quad (\text{loc-scale-family})$$

где параметры $\text{loc}, \text{scale} \in \mathbb{R}$ для вещественнозначных случайных величин, и $\text{loc} \in \mathbb{R}^d$, $\text{scale} \in \mathbb{R}^{d \times d}$ для векторозначных случайных величин. Примером такого семейства является семейство нормальных распределений $\mathcal{N}(\mu, \sigma)$, определяемых равенством (**normal-family**).

$$\mu + \sigma \cdot \xi, \quad \xi \sim \mathcal{N}(0, 1), \quad \text{т.е. } f_{\xi}(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \quad (\text{normal-family})$$

Более общим понятием является понятие семейства замкнутого относительно действия группы, см. [23] и [28].

Другим, в некотором смысле ортогональным, понятием является понятие экспоненциального семейства распределений [3]. Параметрическое семейство распределений $\{\mathbb{P}_{\theta}\}_{\theta \in \Theta}$ относится к экспоненциальному типу, если плотности (или функции вероятностей) которых можно записать в виде (**exp-family**)⁴.

$$f(\mathbf{x}|\theta) = \exp(\langle \mathbf{T}(\mathbf{x}), \vec{\eta}(\theta) \rangle) + A(\mathbf{x}) + D(\theta) \quad (\text{exp-family})$$

Многие распространенные семейства распределений являются экспоненциальными, см. [27]. Для моделей относящихся к экспоненциальным семействам существует богатая теория оценивания параметров [23]. Большой список параметрических семейств и связывающие их соотношения представлены в [22].

Приведенные выше семейства интересны с точки зрения теоретической статистики. С точки зрения прикладной статистики, интерес представляют распределения, которые допускают гибкость в плане оценивания параметров: так, для нормального распределения два параметра не только локацию и масштаб, но и всю форму распределения, причем такое поведение присуще не только нормальному распределению. Для того чтобы решить эту проблему, было предложено несколько гибких семейств распределений, среди которых широко распространены семейство распределений Пирсона [8] и металогиическое семейство [16].

⁴При этом требуется чтобы множество точек $\mathbf{x}$, в которых плотность отлична от 0, не зависело от параметра θ

Отдельно стоит отметить, что многие параметрические семейства зачастую имеют несколько параметризаций, каждая из которых может быть удобна в том или ином контексте, например в работе [30] приведены четыре параметризации для обобщенного гиперболического распределения. Множество других различных параметризаций для одних и тех же семейств собраны в базе проекта ProbOnto [35].

Преобразования случайных величин

Во многих моделях распределения могут быть составлены из более простых распределений с помощью различных методов. В [3] отмечается что, в контексте статистического вывода, любая модель для данных может рассматриваться как вероятностное распределение. Существует множество комбинировать и преобразовывать вероятностные распределения, интересная практическая реализация этого взгляда доступна в библиотеке Pomegranate, [32]. Ниже рассмотрены основные способы для непрерывных вещественнозначных случайных величин, большинство которых относят к теории алгебры случайных величин, см. [33].

- *Аффинное преобразование.* Если случайная величина ξ имеет распределение с функцией плотности $f_\xi(x)$, то плотность её линейного преобразования $a\xi + b$ описывается равенством (**aff-tr**);

$$f_{a+b\xi}(y) = \frac{1}{|a|} f_\xi\left(\frac{y-b}{a}\right), \quad a \neq 0. \quad (\text{aff-tr})$$

- *Биективное преобразование.* Плотность распределения случайной величины $g(\xi)$, где g — строго монотонная функция, определяется через обратную функцию $g^{-1}(y)$ с помощью равенства (**bij-tr**).

$$f_{g(\xi)}(y) = f_\xi(g^{-1}(y)) \cdot \left| \frac{d}{dy} g^{-1}(y) \right|. \quad (\text{bij-tr})$$

Уравнение (**aff-tr**) является частным случаем (**bij-tr**). Для немонотонных функций распределение вычисляется с использованием разбиения на участки монотонности;

- *Распределение суммы независимых случайных величин.* Если случайные величины ξ_1 и ξ_2 независимы⁵, то плотность суммы $\xi_1 + \xi_2$ вычисляется с помощью свёртки (**sum-rv**).

$$(f_{\xi_1} \oplus f_{\xi_2})(z) = \int_{\mathbb{R}} f_{\xi_1}(x) f_{\xi_2}(z-x) dx. \quad (\text{sum-rv})$$

- *Распределение произведения независимых случайных величин.* Для независимых случайных величин ξ_1 и ξ_2 , плотность их произведения $\xi_1 \cdot \xi_2$ задается с помощью мультипликативной свёртки (**prod-rv**).

$$(f_{\xi_1} \odot f_{\xi_2})(z) = \int_{\mathbb{R}} \frac{1}{|x|} f_{\xi_1}(x) f_{\xi_2}\left(\frac{z}{x}\right) dx. \quad (\text{prod-rv})$$

Другие две важные операции возникают при работе со случайными векторами — маргинализация (взятие проекции) и вычисление порядковых статистик. Умение вычислять такие преобразования позволяет получать точные оценки качества в некоторых моделях статистического вывода.

- *Проекция.* Для случайного вектора $\xi = (\xi_1, \xi_2, \dots, \xi_d)$ (с возможно зависимыми компонентами) с функцией распределения $F_\xi(\mathbf{x})$, функция распределения случайного подвектора $(\xi_{i_1}, \dots, \xi_{i_k})$, описывается пределом (**proj-tr**)

$$F_{\text{Pr}(\xi; i_1, \dots, i_k)}(x_{i_1}, \dots, x_{i_k}) = \lim_{\substack{x_i \rightarrow +\infty \\ i \neq i_1, \dots, i_k}} F_\xi(x_1, \dots, x_d) \quad (\text{proj-tr})$$

- *Длины и углы.* Как правило, под случайным вектором понимается случайный элемент $\mathbb{R}^d$ представимый своими координатами. Однако, в некоторых ситуациях (см. например [13]) куда удобнее оперировать со сферическими или другими координатами. В этом случае работает многомерный аналог формулы (**bij-tr**).

Отдельно стоит отметить что важную роль играют порядковые статистики (см. [24]). Если компоненты случайного вектора $\xi = (\xi_1, \xi_2, \dots, \xi_d)$ независимы и одинаково распределены с плотностью распределения $f(x)$ и функцией распределения $F(x)$, то их порядковые статистики $\xi_{(1:d)} \leq \xi_{(2:d)} \leq \dots \leq \xi_{(n)}$ имеют плотности, описываемые равенством (**ord-stat**).

$$f_{\xi_{(k)}}(x) = \frac{n!}{(k-1)!(n-k)!} [F(x)]^{k-1} [1-F(x)]^{n-k} f(x). \quad (\text{ord-stat})$$

⁵Т.е. $\mathbb{P}(\xi_1 \in B_1 \text{ и } \xi_2 \in B_2) = \mathbb{P}(\xi_1 \in B_1) \cdot \mathbb{P}(\xi_2 \in B_2)$ верно для всех $B_1, B_2 \subset \mathbb{R}$, являющихся борелевскими

Помимо трансформации одного распределения в другое и алгебраических операций над распределениями, еще одним способом образования сложного распределения из нескольких простых является образование *смесей*. Смеси используются для построения сложных вероятностных моделей, в которых присутствуют скрытые параметры и широко применяются в кластерном анализе и для изучения ядерных оценок плотности. Можно выделить два класса смесей: дискретные и непрерывные.

- *Дискретная смесь*. Под дискретной смесью подразумевают комбинацию конечного числа распределений, каждое из которых взвешено определённым коэффициентом. Функция распределения дискретной смеси случайных величин $\xi_1, \dots, \xi_n$ с весами $\vec{w} = (w_1, \dots, w_n)$ задаётся равенством (**dmix**);

$$F_{\text{mix}(\vec{w}; \xi)}(x) = \sum_{i=1}^n w_i F_{\xi_i}(x), \quad \sum_{i=1}^n w_i = 1 \quad (\text{dmix})$$

- *Непрерывная смесь* является непрерывным аналогом дискретной смеси. Пусть $F(x | \theta)$ семейство плотностей, зависящее от параметра $\theta \in \Theta \subset \mathbb{R}^n$. Если на Θ задано некоторое распределение параметров с плотностью $\omega(\theta)$, непрерывная смесь плотностей $F(x | \theta)$ определяется равенством (**cmix**)

$$F_{\text{mix}(\omega; F_{\xi}(\cdot | \theta))}(x) = \int_{\Theta} F_{\xi}(x | \theta) \cdot \omega(\theta) d\theta, \quad (\text{cmix})$$

В данном случае параметр θ рассматривается как случайная величина с заданным распределением.

Общая теория смесей в абстрактном случае и конкретные примеры приведены в работе [6].

В приложениях часто возникают понятия цензурированных и урезанных распределений [9]. Для распределения $\mathbb{P}_{\xi}$ случайной величины ξ принимающей вещественные значения, урезанным называется условное распределение, определяемое равенством (**truncated-dist**).

$$\mathbb{P}_{\text{Truncated}(\xi, L, R)}(B) = \frac{\mathbb{P}_{\xi}([L; R] \cap B)}{\mathbb{P}_{\xi}([L; R])} \quad (\text{truncated-dist})$$

Распределение (**truncated-dist**) это условное распределение ξ если априори известно, что значение ξ лежит в отрезке $[L; R]$. В свою очередь, цензурированным на отрезке $[L; R]$ распределением называется распределение случайной величины, определяемой равенством (**censored-dist**).

$$\text{.Censored}(\xi, L, R) = \begin{cases} L & \xi < L \\ \xi & L \leq \xi \leq R \\ R & R < \xi \end{cases} \quad (\text{censored-dist})$$

Такие распределения часто возникают в задачах регрессионного анализа, см. [5].

Числовые характеристики вероятностных распределений

Для анализа моделей важную роль играют не только функциональные, но и числовые характеристики распределений. Согласно [9], [36], для случайной величины ξ со значениями из $\mathbb{R}$ можно выделить следующие характеристики.

- Меры центральной тенденции, описывающие, вокруг какого значения сконцентрированы реализации случайной величины.

- *Математическое ожидание*. Для случайной величины ξ , её математическое ожидание определяется равенством (**mean**);

$$\mathbb{E}[\xi] = \int_{\mathbb{R}} x \mathbb{P}_{\xi}(dx) \quad (\text{mean})$$

- *Медиана*. Для случайной величины ξ , её медиана определяется как множество всех значений m , таких что $F_{\xi}(m) = \frac{1}{2}$, иначе говоря, медиана определяется равенством (**med**).

$$\text{med}[\xi] = F_{\xi}^{-1}\left(\frac{1}{2}\right) \quad (\text{med})$$

В некоторых случаях, в качестве медианы берут какое-то конкретное значение из множества $F_{\xi}^{-1}(\frac{1}{2})$, такое значение называется *точной медианой* [36];

- *Мода*. Для случайной величины ξ её мода определяется равенством (**mode**)

$$\text{mode}[\xi] = \underset{\mathbb{R}}{\operatorname{argmax}} f_{\xi}(x) \quad (\text{mode})$$

где $f_{\xi}(x)$ это функция вероятности, если ξ — дискретная случайная величина, и плотность, если ξ непрерывная случайная величина.

- Меры разброса (иногда меры рассеивания) указывают на склонность величины отклоняться от своего центрального значения.

- Дисперсия и среднеквадратичное отклонение определяются равенствами (**std**) и (**std**) соответственно;

$$\mathbb{D}[\xi] = \mathbb{E}[(\xi - \mathbb{E}[\xi])^2]; \quad (\text{var})$$

$$\text{std}[\xi] = \sqrt{\mathbb{D}[\xi]} \quad (\text{std})$$

- Среднее абсолютное отклонение определяется равенством (**mad**).

$$\text{mad}[\xi] = \mathbb{E}[|\xi - \mathbb{E}[\xi]|] \quad (\text{mad})$$

- Межквартильный размах определяется равенством (**iqr**).

$$\text{IQR}[\xi] = \omega_\xi(0.75) - \omega_\xi(0.25) \quad (\text{iqr})$$

- Меры скоса—мера асимметрии распределения относительно среднего значения.

- Коэффициент скоса определяется равенством (**skew**);

$$\text{skew}[\xi] = \frac{\mathbb{E}[(\xi - \mathbb{E}[\xi])^3]}{\text{std}^3[\xi]} \quad (\text{skew})$$

- Коэффициент скоса Пирсона определяется равенством (**pskew**);

$$\text{Skew}^P[\xi] = \frac{\mathbb{E}[\xi] - \text{med}[\xi]}{\text{mad}[\xi]} \quad (\text{pskew})$$

- Обобщенный коэффициент скоса Грюневельда определяется равенством (**pskew**).

$$\gamma(u) = \frac{\omega_\xi(1-u) + \omega_\xi(u) - 2\omega_\xi(\frac{1}{2})}{\omega_\xi(u) - \omega_\xi(1-u)} \quad \frac{1}{2} < u < 1 \quad (\text{qskew})$$

- Меры эксцесса и тяжести хвостов

- Коэффициент эксцесса измеряет степень остроты вершины распределения и определяется равенством (**kurt**).

$$\text{kurt}[\xi] = \frac{\mathbb{E}[(\xi - \mathbb{E}[\xi])^4]}{\text{std}^4[\xi]} - 3 \quad (\text{kurt})$$

- Квантильный коэффициент эксцесса является квантильным аналогом стандартного коэффициента эксцесса [31] и определяется равенством (**qkurt**);

$$\kappa(u, v) = \frac{\omega_\xi(1-u) - \omega_\xi(u)}{\omega_\xi(v) - \omega_\xi(u)}, \quad 0 < u < v < \frac{1}{2} \quad (\text{qkurt})$$

- Экспонента хвоста определяется для распределений, функция выживания которых убывает согласно степенному закону, как число α при котором верна асимптотическая эквивалентность (**tail-idx**).

$$S_\xi(x) \sim x^{-\alpha}, \quad x \rightarrow \infty. \quad (\text{tail-idx})$$

В общей ситуации отдельно определяется индекс для левого хвоста и для правого хвоста.

Также, для случайной величины ξ и натурального числа $n \in \mathbb{N}$ определены моменты, центральные моменты (m_n и μ_n соответственно в равенстве (**moment**)), абсолютные моменты, абсолютные центральные моменты (v_n и ν_n в равенстве (**abs-moment**)) и факториальные моменты (κ_n в равенстве (**fact-moment**)) порядка n . На основе этих характеристик можно производить оценку параметров распределения.

$$m_n = \mathbb{E}[\xi^n] \quad \mu_n = \mathbb{E}[(\xi - \mathbb{E}[\xi])^n] \quad (\text{moment})$$

$$v_n = \mathbb{E}[|\xi|^n], \quad \nu_n = \mathbb{E}[|\xi - \mathbb{E}[\xi]|^n] \quad (\text{abs-moment})$$

$$\kappa_n = \mathbb{E}[\xi(\xi-1)(\xi-2)\dots(\xi-n+1)] \quad (\text{fact-moment})$$

Обобщением моментов являются так называемые L -моменты [15] и их квантильные аналоги LQ -моменты [26].

Другое семейство числовых характеристик приходит из области теории информации, см. например монографию [19]. Далее, подразумевается что ξ необязательно вещественнозначная случайная величина, со значениями из некоторого пространства $\mathcal{X}$ и под плотностью подразумевается плотность в смысле (**pdf**).

- *Энтропия* распределения с плотностью p относительно меры μ определяется равенством (entr)

$$H_r(p) = - \int_{\mathcal{X}} p(x) \log_r p(x) d\mu \quad (\text{entr})$$

- *Кросс-энтропия* из распределения с плотностью p в распределение с плотностью q определяется равенством (entr)

$$CE_r(p||q) = - \int_{\mathcal{X}} q(x) \log_r p(x) d\mu \quad (\text{cross-entr})$$

- *KL-дивергенция* является мерой расхождения между двумя распределениями с плотностями p и q и определяется равенством (kl-div)

$$\mathcal{D}(p||q) = - \int_{\mathcal{X}} q(x) \log_r \frac{p(x)}{q(x)} d\mu \quad (\text{kl-div})$$

Методы из теории информации активно применяются в статистике, см. например [1]. В частности, зачастую рассматривают обобщенный вариант KL-дивергенции — *f-дивергенцию*, которая определяется для любой выпуклой функции $f: \mathbb{R}_+ \rightarrow \mathbb{R}$ равенством (f-div).

$$\mathcal{D}_f(p||q) = \int_{\mathbb{R}} q(x) f\left(\frac{p(x)}{q(x)}\right) dx \quad (\text{f-div})$$

С информационными характеристиками тесно связана информация Фишера. Для параметрического семейства плотностей $f(x; \theta)$, $\theta \in \Theta \subseteq \mathbb{R}$, информация Фишера это функция от параметра, задаваемая равенством (FI).

$$\mathcal{I}(\theta) = \mathbb{E} \left[\left(\frac{\partial}{\partial \theta} \log f_{\xi}(x; \theta) \right)^2 \right] \quad (\text{FI})$$

Способы моделирования вероятностных распределений

Для вычисления метрик качества и моделирования поведения вероятностных систем необходимо уметь производить стохастическое моделирование случайных величин. Согласно [14], для генерации выборок можно выделить три основных метода:

- *Метод обратного преобразования*, который базируется на том факте, что для случайной величины ξ с квантильной функцией $\omega_{\xi}(p)$, распределение случайной величины $\omega_{\xi}(U)$, $U \sim \mathcal{U}(0; 1)$ будет совпадать с распределением ξ .
- *Метод декомпозиции*, который используется для генерации выборок из смешанных распределений. Общая идея заключается в том, что сначала генерируется значение параметра (например номер кластера), а затем уже случайная величина при условии зафиксированного значения параметра.
- *Метод отбора (rejection sampling)*, который используют когда предыдущие два метода не могут быть использованы. Этот метод значительно медленнее предыдущих и при его использовании возникает много нюансов, но для его использования необходим доступ только к плотности распределения.

При этом дискретные распределения требуют отдельного рассмотрения. Существуют также методы основанные на методе отбора, для генерации выборок из распределений заданных с помощью интегральных преобразований. При этом, нередки ситуации, плотность распределения $f_{\xi}(x)$ известна только с точностью до нормализующей константы или если требуется производить генерацию в сложных или многомерных пространствах. Для таких случаев разработаны методы на основе марковских цепей Монте-Карло (MCMC). Суть метода заключается в том, что на пространстве реализаций надо завести некоторое случайное блуждание, которое в пределе будет давать желанное распределение, детали см. например в книге [25].

Примеры вероятностных моделей в PySATL

В заключение этого раздела отметим, что поддержка работы с представленными ранее объектами является необходимой для PySATL в рамках существующих и будущих проектов. Пакет **MPeSt**⁶ использует различные числовые характеристики, такие как L-моменты для оценок параметров в моделях смеси.

Пакет **NMVMestimation**⁷ занимается специальными видами непрерывных смесей и использует различные интегральные преобразования. Библиотека **Experiment**⁸ использует базовые характеристики распределений для оценок мощностей статистических тестов методом Монте-Карло. С помощью арифметики распределений можно будет получать точные распределения статистик используемых при проверке гипотез. В ближайшем будущем планируется начать разработку библиотек для регрессионного анализа и оценки параметров, где также широко потребуются использование различных свойств и характеристик распределений.

⁶<https://github.com/PySATL/MPeSt>

⁷https://github.com/PySATL/PySATL_NMVM_Module

⁸<https://github.com/PySATL/pysatl-experiment>

Заинтересованные лица

Настоящий раздел описывает различные типы заинтересованных сторон, которым будет интересен данный документ, а также их потенциальные опасения/запросы связанные с дизайном ядра.

Мы отметим, что представленные здесь стороны/лица необязательно относятся к пользователям, которые взаимодействуют с системой.

3.1. Разработчики ядра PySATL

Это лица непосредственно принимающие участие в написании кода для ядра. Для них приоритетным является ряд возможностей:

- Добавление новой функциональности или оптимизация уже существующей;
- Покрытие кодовой базы тестами; в частности регрессионными тестами;
- Сопровождение существующей кодовой базы;
- Расширение коллектива разработчиков;

1. Добавление новой функциональности Разработчики ядра расширяют и модифицируют библиотеку в контексте следующих задач:

- Добавление новых параметрических семейств/новых операций над распределениями;
- Добавление новых числовых и/или функциональных характеристик распределений;
- Добавление новых численных методов для вычисления характеристик распределений;
- Добавление новых численных методов для вычисления операций над распределениями;

2. Тестирование Помимо расширения функциональности, разработчики ядра покрывают кодовую базу тестами; Им необходимо иметь ряд механизмов для:

- автоматического тестирования новой функциональности, касающейся свойств добавляемых распределений/семейств/операций
- создания регрессионных тестов для функциональности, оперирующей с псевдо-случайными данными

Пример. При создании нового семейства распределений разработчик указывает что все его представители имеют носитель на отрезке $[0; 1]$. Это свойство распределения которое можно проверить например сгенерировав выборку из данного распределения и проверив что все элементы выборки лежат в этом отрезке.

Пример. Валидационные примеры использования статистических процедур часто служат основой для регрессионных тестов этих самых процедур. Библиотека должна обеспечивать воспроизводимость генерации выборок.

3. Сопровождение кодовой базы и расширения коллектива разработчиков

Сопровождение кодовой базы и расширение коллектива разработчиков осуществляется за счет нескольких инструментов

- Архитектурная документация; настоящий документ является её частью. В частности, так как ядро планируется активно использовать в других библиотеках, необходимо явно отразить какие части системы являются публично доступными
- Техническая и пользовательская документация; наличие качественной документации, содержащей примеры использования и описывающей роли компонент системы

3.2. Разработчики других библиотек в PySATL

К этой категории относятся все разработчики PySATL, которые либо не принимают непосредственного участия в разработке ядра, либо делают это эпизодически, добавляя функциональность под конкретные нужды проектов (в последнем случае они относятся к первой категории заинтересованных лиц). Их основные интересы по отношению к ядру состоят в следующем.

1. Простота интеграции ядра в другие библиотеки PySATL;
2. Конфигурируемость ядра для типичных сценариев использования;

1. Простота интеграции ядра в другие библиотеки PySATL

Сейчас PySATL активно использует связку Numpy/SciPy для большинства математических вычислений; Это означает что при переходе на ядро, во всех местах использующих эту связку, замена не должна вызвать осложнений. В частности ядро

- Не должно обязывать разработчика других библиотек к конфигурации численных методов, использующихся в ядре;
- Должно иметь всю ту же функциональность, что и модуль `statistics` библиотеки SciPy;
- Должно предоставлять унифицированный интерфейс для прочих математических функций;

Здесь надо пояснить что такое прочие математические функции. При реализации различных оценок, иногда возникают довольно нетривиальные объекты (так `rysatl-nmvm` использует многочлены Белла для вычислений, а `mpest` - различные методы оптимизации). Так как существует множество математических библиотек, которые могут предоставлять данные возможности, то:

1. Возможно, конечный пользователь библиотеки (не обязательно ядра) захочет чтобы использовались какие-то конкретные математические пакеты (что очень может быть в случае интегрирования/оптимизации)
2. Наличие единого интерфейса для математических утилит не приведет к тому что внутри разных библиотек (или даже одной библиотеки) используются разные математические пакеты

К тому же, многие из математических пакетов, предоставляющих редкие, но нужные функции, являются либо проприетарными, либо с вирусными лицензиями. Использование и тех, и других, не подходит под лицензионные ограничения; создание единого интерфейса для математических утилит позволяет распространять PySATL под MIT лицензией, используя вирусную лицензию только для реализации этих интерфейсов. Это значительно позволит сократить ресурсы на прототипирование различных библиотек на базе ядра.

2. Конфигурируемость ядра для типичных сценариев использования

Различные библиотеки проекта PySATL предъявляют различные, зачастую противоречивые, требования к вычислительным характеристикам ядра. Библиотека должна предоставлять механизмы тонкой настройки без необходимости модификации её исходного кода. Это включает в себя:

- **Выбор численных методов:** Возможность выбора конкретной реализации алгоритма (например, для вычисления интеграла или оптимизации) в зависимости от требований к точности, скорости или устойчивости. Библиотека `experiment` может требовать максимальной скорости для генерации больших выборок, в то время как `mpest` — максимальной надёжности и точности для сходимости алгоритмов оценки параметров.
- **Управление вычислительными ресурсами:** Настройка параметров, влияющих на производительность и использование памяти. Например:
 - Установка допусков (`tolerances`) для итеративных алгоритмов;
 - Задание лимитов на количество итераций;
 - Управление размером кэшей для повторных вычислений.
- **Стратегии семплирования:** Конфигурация генераторов псевдослучайных чисел (ГПСЧ), включая:
 - Выбор конкретного алгоритма ГПСЧ (например, Mersenne Twister, PCG64);
 - Установка начального значения (`seed`) для обеспечения воспроизводимости результатов;
 - Управление параллельными потоками генерации для эффективного использования многоядерных систем.
- **Политики обработки ошибок:** Возможность настройки реакции на исключительные ситуации — от строгого прерывания вычисления с выводом подробной ошибки до возврата специальных значений (`NaN`, `inf`) или использования fallback-алгоритмов для обеспечения отказоустойчивости.
- **Абстракция математического бэкэнда:** Единый интерфейс для низкоуровневых математических операций (линейная алгебра, специальные функции, оптимизация), позволяющий подменять реализации без изменения кода, зависящего от ядра. Это гарантирует, что библиотеки `nmvm` и `mpest` будут использовать согласованный `computational stack`, конфигурируемый централизованно.

3.3. Руководители проекта PySATL

Данная категория включает лиц, ответственных за планирование развития проекта в целом, распределение ресурсов и координацию между рабочими группами. Их интересы сфокусированы на стратегических аспектах разработки и эксплуатации ядра, а также на его интеграции в общую экосистему PySATL.

- Представление о планах разработки ядра и его функциональности на каждом этапе;
- Гарантии, предоставляемые ядром;
- Эффективное использование ресурсов и соблюдение сроков;
- Минимизация рисков, связанных с техническим долгом и зависимостями.

1. Планы разработки и функциональность

Руководителям проекта необходимо иметь чёткое и актуальное представление о состоянии и roadmap ядра для принятия обоснованных решений. Это включает:

- Соответствие фактической функциональности ядра заявленным планам и требованиям других подсистем PySATL;
- Наличие документации, достаточной для оценки готовности компонент к интеграции;
- Понятный и измеримый прогресс разработки, позволяющий оценивать выполнение этапов.

2. Гарантии, предоставляемые ядром

Поскольку ядро является фундаментальным компонентом всего проекта, от его качества и стабильности зависит работа всех надстроенных над ним библиотек. Руководители заинтересованы в наличии гарантий, обеспечиваемых за счёт:

- Всестороннего тестирования, включая модульные, интеграционные и регрессионные тесты;
- Чёткого определения и поддержания публичных интерфейсов (API), что обеспечивает стабильность для потребителей ядра;
- Предсказуемой политики управления версиями, минимизирующей обратно несовместимые изменения;
- Исчерпывающей документации, снижающей риски misinterpretation API и его неправильного использования.

3. Эффективность использования ресурсов и сроки

Процесс разработки ядра должен быть организован таким образом, чтобы:

- Архитектурные решения способствовали повторному использованию кода и сокращению избыточности;
- Существовали механизмы для быстрого прототипирования и валидации новых идей без угрозы нарушения стабильности основной codebase;
- Соблюдались согласованные сроки поставки критически важной для других подсистем функциональности.

4. Управление рисками

Ключевым аспектом является проактивное выявление и снижение потенциальных рисков, в частности:

- Контроль над внешними зависимостями, их лицензионной чистотой и совместимостью;
- Минимизация накопления технического долга за счёт регулярного рефакторинга и применения современных практик разработки;
- Наличие плана по обеспечению долгосрочной поддерживаемости и расширяемости кодовой базы.

3.4. Инженеры-исследователи

К данной категории относятся специалисты, применяющие ядро PySATL для решения прикладных задач: программисты, математики, статистики и исследователи в области численных методов. Они используют систему как инструмент для реализации и проверки научных гипотез, прототипирования алгоритмов и проведения вычислительных экспериментов.

- Поддержка сложных операций над распределениями, смесями и иерархическими моделями;
- Бенчмаркинг новых алгоритмов и сравнение с существующими аналогами;
- Гибкость и выразительность для описания нестандартных вероятностных моделей;
- Воспроизводимость и контролируемость вычислительных экспериментов.

1. Поддержка сложных операций и моделей

Инженеры-исследователи работают с комплексными вероятностными структурами, выходящими за рамки стандартных распределений. Их интерес заключается в возможности:

- Конструировать составные распределения (смеси, свёртки, произведения);
- Определять иерархические (многоуровневые) вероятностные модели;
- Применять нелинейные функциональные преобразования к распределениям;
- Работать с усечёнными, цензурированными или модифицированными распределениями.

2. Бенчмаркинг и валидация алгоритмов

Для данной аудитории критически важна возможность оценивать эффективность и корректность реализуемых методов:

- Наличие встроенных средств для замеров производительности (профилирования) отдельных операций;
- Возможность сравнивать численные результаты, полученные с помощью ядра, с эталонными реализациями или аналитическими выкладками;
- Доступ к низкоуровневым параметрам численных методов (таким как точность, допуски, критерии останова) для тонкой настройки и исследования сходимости алгоритмов.

Пример. Исследователь, разрабатывающий новый метод оценки параметров, должен иметь возможность сгенерировать набор синтетических данных из известного распределения, применить свой метод, сравнить полученные оценки с истинными значениями параметров и измерить время выполнения алгоритма, обеспечивая при этом полную воспроизводимость эксперимента.

3. Гибкость и выразительность

Система должна предоставлять достаточный уровень абстракции для лаконичного и интуитивно понятного выражения математических концепций:

- Декларативный способ задания моделей, приближенный к математической нотации;
- Возможность легко комбинировать примитивы ядра для создания специализированных конструкций без необходимости написания большого объема boilerplate-кода;
- Доступ к символьным представлениям преобразований для анализа и оптимизации выражений до их численного вычисления.

4. Воспроизводимость и контролируемость

Так как работа инженера-исследователя носит экспериментальный характер, ядро должно предоставлять инструменты для обеспечения строгости научного процесса:

- Детерминированная работа генератора псевдослучайных чисел с возможностью сохранения и восстановления состояния (seed);
- Логирование ключевых этапов вычислений и принятых решений (например, в алгоритмах оптимизации);
- Чёткое разделение между абстрактной спецификацией модели и конкретным численным методом её вычисления.

Ключевые требования, определяющие архитектуру

4.1. Сценарии использования системы

todo; see [ISSUE](#)

4.2. Технические ограничения и качественные требования

4.2.1. Технические ограничения

В данном разделе перечислены внешние технические ограничения, влияющие на архитектуру и реализацию системы. Эти ограничения задаются проектом PySATL или внешними условиями и не подлежат изменению в рамках проектирования ядра.

- *Язык реализации:* система должна предоставлять основной пользовательский интерфейс на языке Python версии 3.x. Допускается использование других языков программирования (например, C/C++) во внутренней реализации с целью оптимизации вычислений, при условии полной прозрачности интерфейса для Python-пользователя.
- *Лицензирование:* система должна распространяться под MIT-лицензией. Использование копиLEFT-лицензий (например, GPL) запрещено.
- *Кроссплатформенность:* система должна работать на основных операционных системах (Linux, macOS, Windows) без необходимости установки платформозависимых компонентов или привязки к конкретной среде исполнения.
- *Вычислительные ограничения:* система не должна требовать наличия GPU или специализированного оборудования. Все вычисления должны корректно выполняться на CPU без дополнительных зависимостей.

4.2.2. Качественные характеристики системы

Данный раздел описывает нефункциональные требования, оказывающие влияние на архитектуру, проектные решения и организацию компонентов системы.

4.2.2.1. Расширяемость

Система должна обеспечивать возможность добавления новых распределений, семейств и преобразований без необходимости изменения существующего кода. Расширение должно происходить через зарегистрированные интерфейсы или стандартные механизмы расширения Python.

4.2.2.2. Модульность и интерфейсы:

Система должна быть реализована в виде набора Python-модулей с чётко определёнными публичными интерфейсами. Модули должны быть слабо связаны друг с другом, а взаимодействие между ними должно происходить исключительно через открытые контракты. Пакет в целом должен также иметь публичный интерфейс для взаимодействия с пользователем и другими подсистемами PySATL. Внутренние и внешние интерфейсы могут быть различны.

4.2.2.3. Совместимость

Система должна быть совместима с остальными компонентами PySATL. Переход на новое ядро должен быть максимально прозрачным для пользователя и не требовать значительных изменений в существующем коде или логике пакетов, использующих ядро.

4.2.2.4. Тестируемость

Архитектура должна способствовать покрытию тестами ключевой функциональности системы. Для стадии прототипа допускается ограниченное покрытие, однако оно должно быть достаточным для выявления ошибок при изменении поведения или интерфейсов.

4.2.2.5. Устойчивость к ошибкам

Система должна быть устойчива к некорректным пользовательским данным и иметь механизм информирования об ошибках. Поведение при ошибках должно быть контролируемым и предсказуемым, включая диагностику нарушений интерфейсов, типов и других контрактов.

4.2.2.6. Документируемость и визуализация:

Архитектура модулей, их интерфейсы и взаимосвязи должны быть представлены в виде UML-диаграмм и сопровождаться соответствующей технической документацией. Диаграммы должны отражать как общую структуру системы, так и реализацию ключевых компонентов и связей.

4.3. Ключевые функциональные требования

В данном разделе изложены функциональные требования к системе, определяющие её основную вычислительную модель, интерфейсы и возможности расширения. Эти требования отражают ожидаемое поведение ядра и набор поддерживаемых операций. Можно выделить следующие группы требований

- (ГФТ1) *Определение распределения*: каждое распределение должно быть определено посредством как минимум одной функциональной характеристики (например, функция плотности или функция распределения), поддерживать генерацию выборки и вычисление логарифма правдоподобия. Распределение должно быть связано с типом случайной величины (дискретной или непрерывной) и типом значений выборки (многомерной или одномерной).
- (ГФТ2) *Функциональные и числовые характеристики*: система должна поддерживать вычисление основных и дополнительных характеристик распределений, включая возможность их добавления пользователем, а также предоставлять гарантии корректности вычислений.
- (ГФТ3) *Семплирование распределений*. Для каждого распределения должна быть доступна генерация выборки. При этом, пользователь должен иметь возможность добавлять свои способы семплирования и настраивать используемый способ семплирования. Система также должна предоставлять механизмы проверки качества семплирования.
- (ГФТ4) *Поддержка основных распределений*: система должна предоставлять встроенную поддержку наиболее распространённых распределений (нормальное, экспоненциальное, биномиальное и др.), а также предусматривать механизм регистрации пользовательских распределений.
- (ГФТ5) *Работа с семействами распределений*: система должна поддерживать задание семейств распределений, определяемых через параметрические пространства. Необходимо обеспечить возможность описания связей между семействами, включая асимптотику и граничные переходы (например, предельные случаи распределений при изменении параметров). Поддержка пользовательских семейств и добавления новых связей обязательна.
- (ГФТ6) *Преобразования распределений*: система должна поддерживать стандартные операции преобразования распределений, включая арифметические операции, функциональные отображения, свёртки, смеси и т.п. Должна быть предусмотрена возможность компоновки и каскадирования таких преобразований.

4.3.1. Требования к числовым и функциональным характеристикам

Под корректным вычислением числовой или функциональной характеристики заданного распределения подразумевается следующее поведение системы:

1. В случае если значение характеристики определено и *всегда* является числом, оно должно вычисляться с предопределённой гарантией точности (например, в виде верхней оценки на относительную погрешность);
2. В случае если значение характеристики *равно* $\pm\infty$, должно выдаваться значение специального типа, представляющего собой абстракцию бесконечности;
3. В случае если значение характеристики *неопределенно*, система должна выдать объект типа NaN. Опционально должно выдаваться предупреждение о том что соответствующее значение не является числом или $\pm\infty$.
4. В случае если значение характеристики *может быть равно* $\pm\infty$ или *может быть неопределенно*, и заранее это невозможно проверить, необходимо предоставлять набор условий, при выполнении которых, с близкой к 1 вероятностью, значение можно считать вычисленным корректно, в смысле п.1;

Полнота:

- Для любого распределения должно быть доступно вычисление всех характеристик моментного/квантильного/информационного типа приведенных в глоссарии;

- Для любого распределения должно быть доступно вычисление всех функциональных характеристик (если они существуют для данного распределения) приведенных в глоссарии.

Гибкость и конфигурируемость:

- При наличии нескольких алгоритмов для расчета числовой/функциональной характеристики данного распределения, должен быть определен алгоритм, который используется по умолчанию;
- При наличии нескольких алгоритмов для расчета числовой/функциональной характеристики пользователь должен иметь возможность указать, какой алгоритм использовать.

Расширяемость:

- Пользователь должен иметь возможность добавлять свои собственные алгоритмы вычисления для уже существующих функциональных/числовых характеристик;
- Пользователь должен иметь возможность добавлять собственные числовые характеристики. В случае если числовая характеристика определена не для всех распределений, пользователь должен иметь возможность явно указать для какого класса распределений определена характеристика;
- Пользователь должен иметь возможность добавлять собственные функциональные характеристики. В случае если функциональная характеристика определена не для всех распределений, пользователь должен иметь возможность явно указать для какого класса распределений определена характеристика.

4.3.2. Требования к семействам распределений

- Для любого семейства распределений должна быть возможность создать новое семейство распределений, получаемое из исходного частичной подстановкой значений параметров;
- Для любого семейства распределений должна быть определена каноническая параметризация;
- Для любого семейства распределений должна быть возможность добавления новой параметризации. При добавлении новой параметризации, пользователь должен указать как параметры добавляемой параметризации выражаются через каноническую параметризацию и наоборот;
- Над семействами необходимо уметь производить операции образования смесей: непрерывных и дискретных;
- Должна быть возможность указать как представители семейства взаимодействуют с представителями других семейств при выполнении операций.

4.3.3. Требования к операциям над распределениями

- Для одномерных непрерывных распределений должны быть доступны операции описанные в параграфе «Преобразования распределений» в глоссарии;
- При наличии нескольких способов вычисления операции, должен быть определён способ вычисления по умолчанию.

4.3.4. Требования к генерации выборок

- Для любого распределения, за исключением относящихся к распределениям геометрических примитивов, должна быть возможность генерации выборок;
- При создании пользовательского распределения, система должна предоставлять методы для генерации выборок без необходимости реализовывать алгоритмы генерации;
- При наличии нескольких алгоритмов генерации выборок, пользователь должен иметь возможность выбирать алгоритм;
- Если алгоритм генерации выборок имеет параметры, то пользователь должен уметь настраивать эти параметры. К числу параметров также относится используемый источник случайных или псевдослучайных чисел.

Избранные архитектурные точки зрения

5.1. Контекст

PySATL Core представляет собой вычислительное ядро экосистемы PySATL. Его назначение — предоставить независимое от конкретных реализаций распределений API для:

- создания распределений из параметрических семейств;
- вычисления функциональных и числовых характеристик по единым контрактам;
- генерации выборок с настраиваемой стратегией сэмплирования;
- расширения системы пользователем без модификации исходного кода ядра.

Внешние акторы и окружение. Ядро используется конечными пользователями, которым требуется вычислять характеристики и строить выборки, и другими пакетами PySATL, которые строят на базе ядра прикладные методы математической статистики. На уровне инфраструктуры ядро опирается на стандартную среду Python и числовой стек (в первую очередь массивы NumPy); при этом архитектурной целью является уменьшение зависимости прикладных пакетов от `scipy.stats` за счёт переноса вычислительной модели в собственные модули.

Граница системы. В рамках ядра фиксируются контракты распределения, семейств, поддержек (support), стратегий вычислений и сэмплирования, а также глобальный реестр преобразований характеристик. Вне границы системы остаются:

- визуализация, статистические тесты и прикладные алгоритмы верхнего уровня;
- конкретные численные методы интегрирования/оптимизации, если они предоставляются внешними библиотеками;
- хранение данных и пользовательские интерфейсы прикладных пакетов.

Точки расширения. Архитектура предполагает, что пользователи и пакеты экосистемы могут:

- регистрировать новые семейства и параметризации;
- добавлять новые аналитические реализации характеристик;
- добавлять новые вычислительные методы и правила вывода в глобальный реестр характеристик;
- подменять стратегии вычислений и сэмплирования для конкретного распределения.

5.2. Композиция

Архитектура PySATL Core организована вокруг набора относительно независимых компонент, объединённых общей вычислительной моделью: характеристики распределения вычисляются через единые контракты, а правила вывода оформляются как расширяемый граф зависимостей. Такая композиция позволяет наращивать систему (новыми семействами, характеристиками, способами вычислений и сэмплирования) без модификации уже реализованных частей.

5.2.1. Состав компонент

На уровне верхней декомпозиции выделяются следующие компоненты (рис. 5.1):

- **families** — описание параметрических семейств распределений, их параметризаций и ограничений, а также фабрика построения конкретных распределений;
- **distributions** — единая «поверхность» доступа к распределению: вычисление характеристик, выбор стратегий, работа с поддержкой (support) и интеграция с реестром характеристик;
- **sampling** — инфраструктура генерации выборок и представления результата (тип выборки), поставляемая как отдельная, переиспользуемая часть ядра;
- **transformations** — компонент описания преобразований распределений и правил пересчёта характеристик при композиции (на момент актуализации проектно значим и рассматривается как целевое расширение).

Внутри компонентов выделены подсистемы, фиксирующие устойчивые точки расширения:

- в **distributions** — **registry**, содержащий глобальный реестр преобразований характеристик и механизмы получения *локального* представления графа для конкретного распределения;
- в **families** — **builtins**, включающий готовые семейства и общие элементы описания параметризаций.

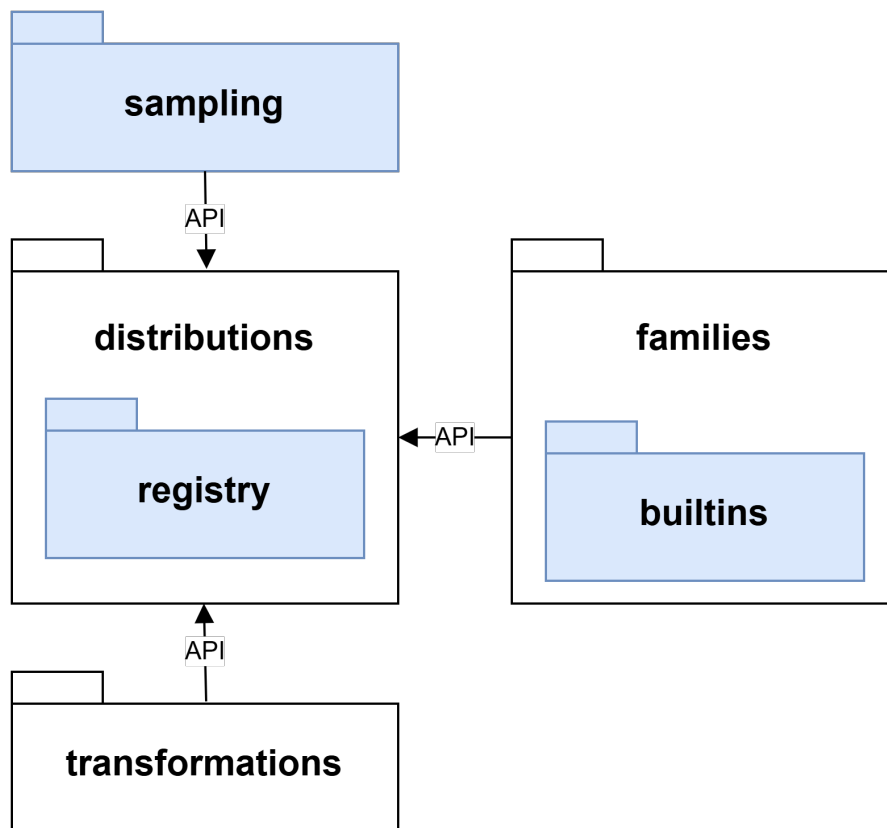


Рис. 5.1: Компоненты ядра и границы публичных API

5.2.2. Роли компонент

families. Компонент отвечает за *порождение* распределений. Семейство задаётся как объект, содержащий набор параметризаций и правила их приведения к базовой форме. Для каждой параметризации могут быть объявлены ограничения на параметры. Результатом работы **families** является объект распределения, который реализует контракт **Distribution** и тем самым бесшовно подключается к вычислительной модели **distributions**.

distributions. Компонент определяет контракт распределения и обеспечивает вычисление характеристик. В частности, он фиксирует:

- *идентификаторы характеристик* (на уровне типов) и единый механизм запроса вычисления;
- *стратегии* вычислений и сэмпирования как взаимозаменяемые политики;
- *поддержку* случайной величины как отдельный объект со своими инвариантами;
- *реестр характеристик* как расширяемый ориентированный граф правил вывода.

Компонент не фиксирует конкретный численный алгоритм «внутри» каждого метода, но задаёт формат его подключения и условия применимости.

sampling. Компонент инкапсулирует генерацию выборок и представление результата. Конкретные алгоритмы сэмпирования поставляются через **SamplingStrategy**; при этом распределение хранит выбранную стратегию как часть своего состояния и делегирует ей операцию **sample**.

transformations. Компонент предназначен для описания преобразований распределений (например, аффинные преобразования, смеси, свёртки) и их влияния на характеристики и поддержку. В целевой архитектуре такие правила должны быть выражены через те же абстракции вычислительных методов, что и «внутренние» преобразования характеристик, и регистрироваться в общем графе. Это обеспечивает единый путь вычисления характеристик как для атомарных распределений, так и для составных.

5.2.3. Поток данных и точки соприкосновения

Типовой поток использования выглядит следующим образом:

1. Пользователь выбирает семейство и задаёт параметры в удобной параметризации.

2. **families** валидирует параметры и приводит их к базовой параметризации.
3. Создаётся объект распределения, предоставляющий **Distribution-API**.
4. При запросе характеристики **distributions** выбирает подходящий метод: либо из аналитически заданных вычислений, либо через вывод по графу реестра; при необходимости применяется кэширование.
5. При запросе выборки распределение делегирует генерацию выборки стратегии **SamplingStrategy**.

Таким образом, граница между компонентами проводится по *контрактам*: **families** формирует корректный экземпляр распределения, а **distributions** обеспечивает вычислительную инфраструктуру и расширяемость через реестр и стратегии.

5.3. Логическая структура

5.3.1. Общий обзор

Логическая структура ядра строится вокруг небольшого набора устойчивых типов и контрактов, которые используются во всех компонентах:

- **протоколы** (`typing.Protocol`) задают интерфейсы **Distribution**, стратегий и вычислительных процедур, позволяя реализовывать их без жёсткой иерархии наследования;
- **типизированные идентификаторы** (например, `GenericCharacteristicName`, `FamilyName`) унифицируют адресацию характеристик и семейств;
- **тип распределения** (`DistributionType`) и связанные с ним признаки (`registry_features`) предоставляют входные данные для ограничений применимости правил вывода характеристик;
- **носитель** (`Support`) описывает множество допустимых значений случайной величины и используется как в семействе (для вывода `support` из параметров), так и в вычислениях (для выбора корректных правил).

На рис. 5.2 приведён обзор ключевых общесистемных типов. Указанные сущности выступают «языком сопряжения» между **families**, **distributions** и будущими **transformations**: правила и стратегии опираются на них, но не зависят от конкретных реализаций распределений.

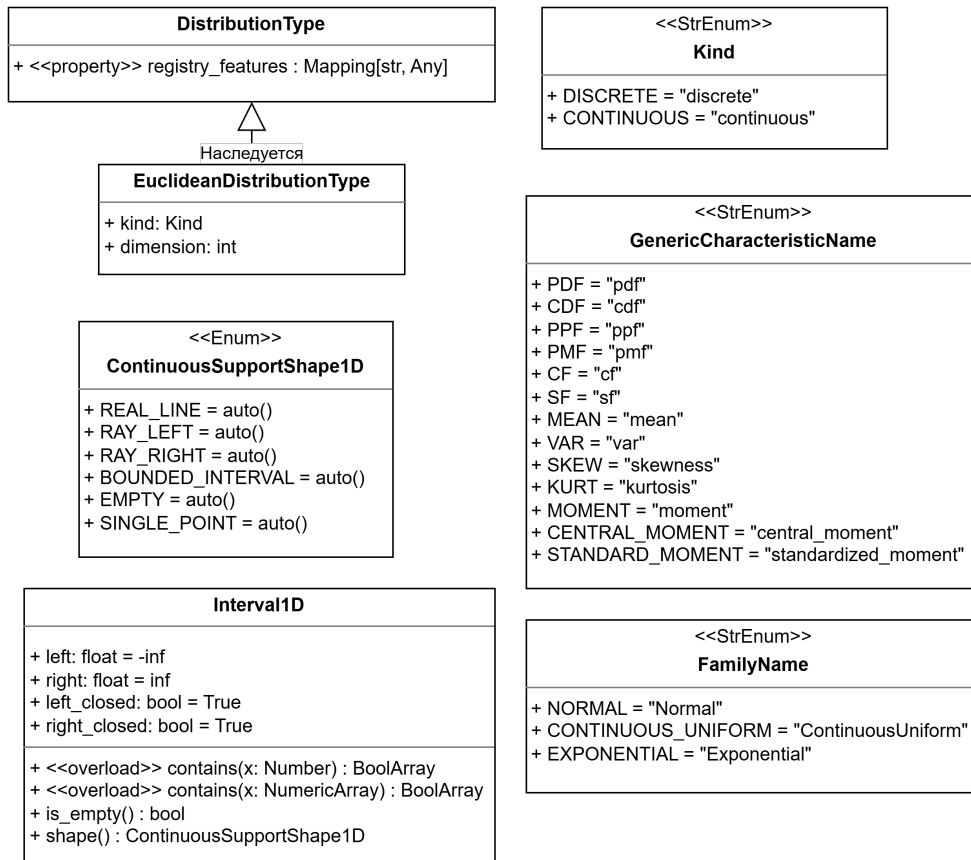


Рис. 5.2: Общие типы и идентификаторы: виды распределений, имена характеристик и семейств, а также базовые представления носителя

5.3.2. Модуль distributions

Компонент `distributions` задаёт единый контракт распределения и инфраструктуру вычислений. Ключевое проектное решение состоит в том, что пользовательский объект распределения предоставляет лишь минимальный набор свойств и методов, а выбор конкретных алгоритмов делегируется стратегиям и глобальному реестру характеристик. Такая схема отделяет описание распределения от правил вывода и обеспечивает расширяемость.

5.3.2.1. Контракт Distribution и стратегии

На рис. 5.3 показан интерфейс `Distribution` в виде протокола Python. Использование протокола позволяет реализовывать распределения как в составе ядра, так и в стороннем коде без принудительного наследования от общего базового класса.

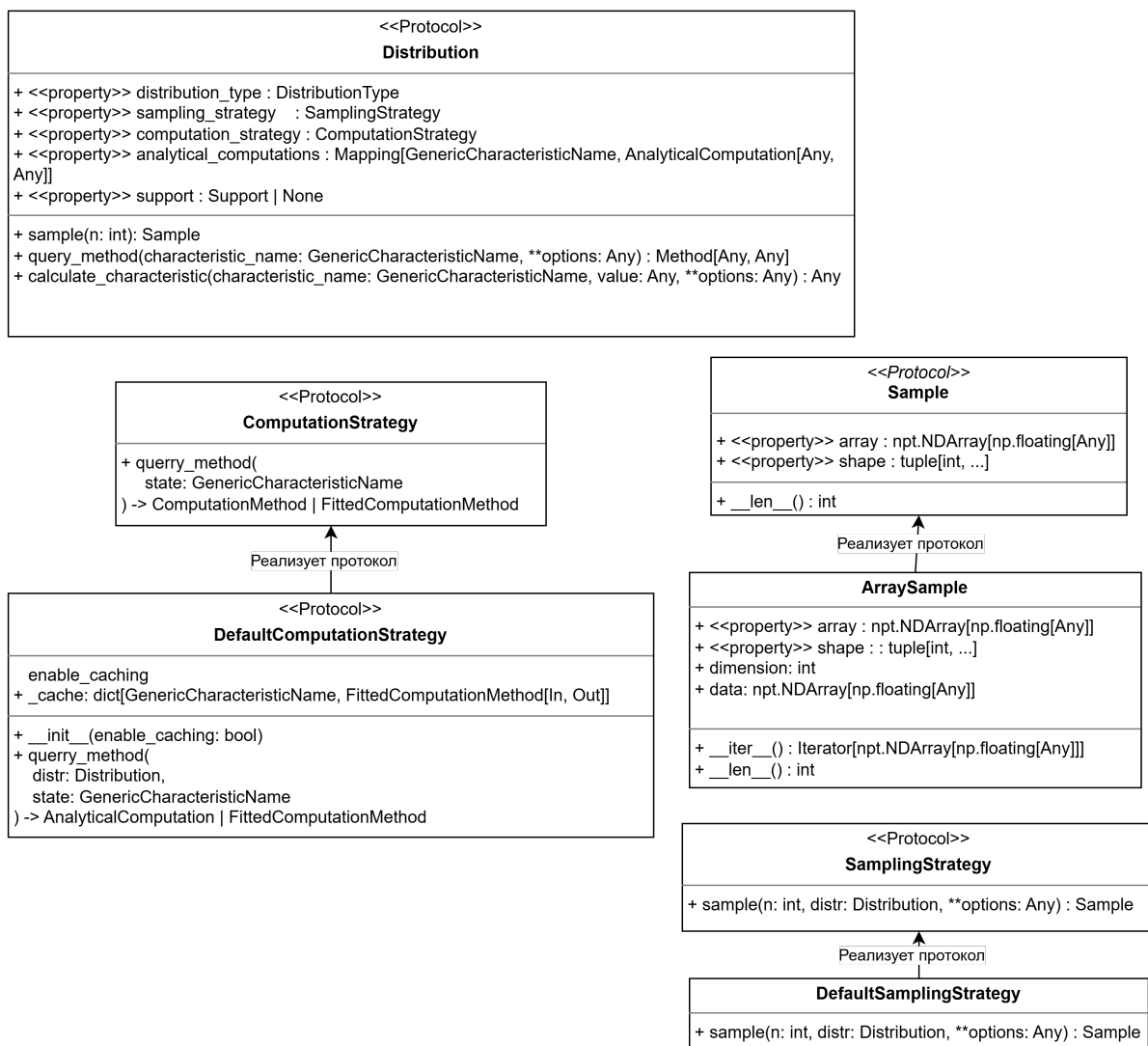


Рис. 5.3: `Distribution` и связанные стратегии вычислений и сэмплирования, а также представление выборки

Интерфейс `Distribution` фиксирует:

- `distribution_type`: тип распределения (см. рис. 5.2), используемый для задания признаков и ограничения применимости правил;
- `support`: объект носителя (`support`), описывающий допустимые значения случайной величины;
- `analytical_computations`: словарь аналитически заданных вычислений характеристик, выступающих «якорями» для вывода остальных;
- `sampling_strategy` и `computation_strategy`: взаимозаменяемые политики генерации выборки и выбора метода вычисления.

Вычисления инициируются через два метода:

- `query_method(...)` возвращает вызываемый объект (метод вычисления) для указанной характеристики;

- `calculate_characteristic(...)` выполняет вычисление немедленно, применяя найденный метод к переданному аргументу.

Такое разделение удобно для переиспользования: один и тот же метод может быть запрошен один раз и затем применён к набору входных значений.

Стратегия вычислений. `ComputationStrategy` отвечает за выбор конкретного способа получения характеристики. Реализация по умолчанию (`DefaultComputationStrategy`) решает две задачи: выбирает, можно ли вернуть аналитическую реализацию напрямую, и, при необходимости, строит вычисление через граф преобразований характеристик, дополняя это кэшированием результатов «подгонки» методов (`fitting`).

Стратегия сэмплирования. `SamplingStrategy` инкапсулирует алгоритм генерации выборки и возвращает объект, реализующий протокол `Sample`. Базовая реализация `DefaultSamplingStrategy` задаёт безопасный путь по умолчанию, а специализированные стратегии могут учитывать особенности распределения, генератора случайных чисел или требования к качеству сэмплирования.

5.3.2.2. Вычислительная модель характеристик

Единая модель вычислений показана на рис. 5.4. В ядре различаются:

- **Computation** — минимальный контракт вычисления конкретной характеристики (`target`) как вызываемого объекта;
- **AnalyticalComputation** — прямое (аналитическое) вычисление, не требующее предварительной подготовки;
- **ComputationMethod** — «шаблон» вычисления, который сначала *подгоняется* под конкретное распределение (`fit`), а затем превращается в **FittedComputationMethod** — вызываемый объект, пригодный для многократного применения.

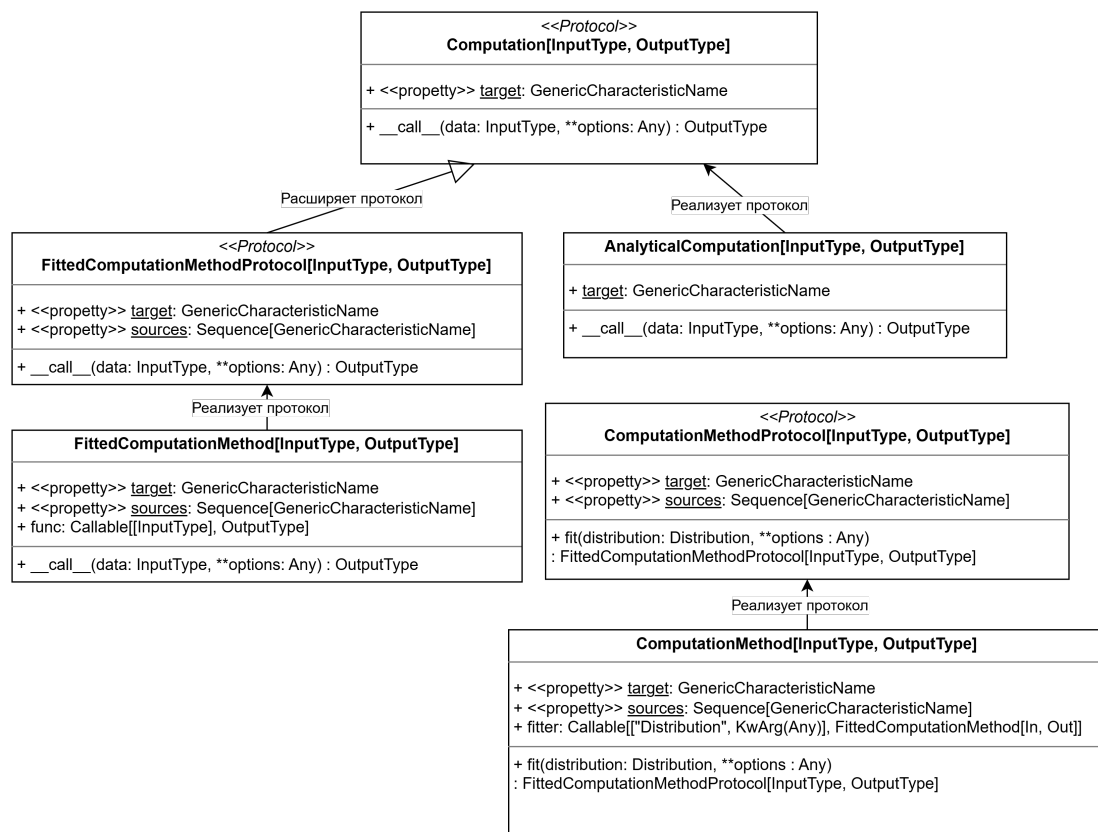


Рис. 5.4: Абстракции вычислений характеристик: аналитические вычисления, методы и подогнанные методы

Ключевое различие между `ComputationMethod` и `FittedComputationMethod` состоит в том, что «подгонка» позволяет захватить параметры распределения, выбранные численные настройки и подготовленные структуры данных (например, сетки или табличные аппроксимации). Это делает кэширование эффективным: стратегия может хранить уже подогнанные методы и не выполнять подготовку повторно.

Поле **sources** фиксирует зависимости: вычисление целевой характеристики (**target**) может требовать значений других характеристик. Эти зависимости используются стратегией вычислений при построении цепочки вывода по графу реестра.

5.3.2.3. Носитель (support)

Носитель случайной величины оформлена как самостоятельный объект. Это позволяет валидировать значения, выражать свойства области определения в правилах вывода, и унифицировать обработку непрерывных и дискретных случаев. На рис. 5.5 показана структура интерфейсов носителей.

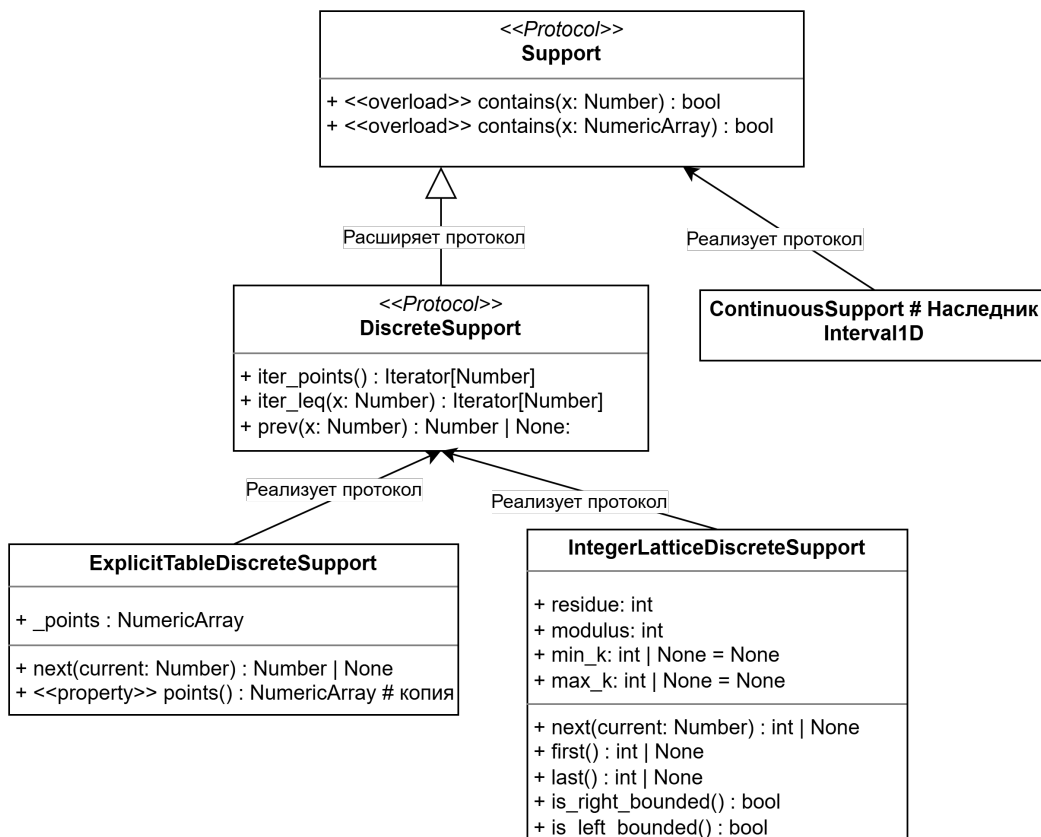


Рис. 5.5: Модель носителя: **Support**, **DiscreteSupport** и базовые реализации

Для непрерывного одномерного случая используется интервальное представление (**Interval1D**), из которого наследуется **ContinuousSupport**. Для дискретного случая предусмотрены две базовые реализации: **ExplicitTableDiscreteSupport** (конечное множество точек) и **IntegerLatticeDiscreteSupport** (целочисленная решётка). Наличие итераторов по точкам и ближайшим значениям позволяет строить численные процедуры, зависящие от структуры **support**.

5.3.2.4. Глобальный реестр характеристик

Центральным механизмом расширения является глобальный синглтон **CharacteristicRegistry** (рис. 5.6). Реестр хранит:

- множество известных характеристик и их классификацию на *definitive/indefinitive*;
- ориентированный граф переходов между характеристиками;
- набор вариантов (label-ов) для одного и того же перехода, позволяющий иметь несколько реализаций;
- ограничения применимости правил (constraints), зависящие от признаков типа распределения и/или экземпляра.

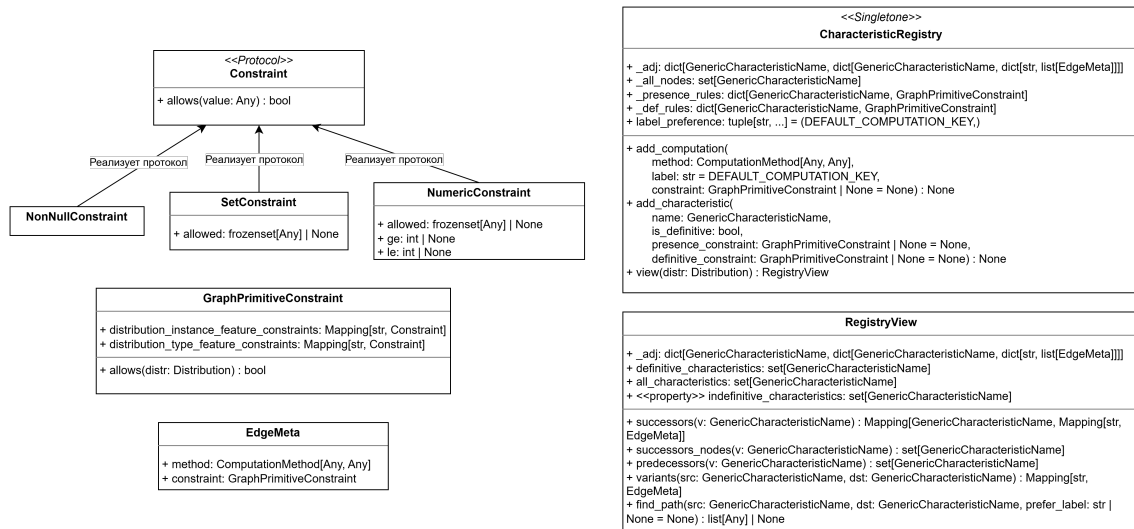


Рис. 5.6: Реестр характеристик: ограничения применимости, метаданные рёбер, глобальный `CharacteristicRegistry` и локальный `RegistryView`

Ограничения применимости. Ограничение задаётся как предикат `allows(...)` и может проверять как свойства распределения, так и произвольные дополнительные признаки, поставляемые через `DistributionType.registry_features`. Составное ограничение `GraphPrimitiveConstraint` позволяет выражать правила вида «данная характеристика/переход присутствует только при выполнении условий на тип распределения и параметры экземпляра».

Локальный вид графа. Метод `CharacteristicRegistry.view(distr)` строит `RegistryView` — представление графа, отфильтрованное под конкретное распределение. В `RegistryView` доступны операции навигации (`successors/predecessors`), перечисление вариантов перехода между двумя узлами и поиск пути `find_path`. Именно `RegistryView` используется стратегией вычислений, что позволяет отделить глобальную конфигурацию реестра от конкретного контекста выполнения.

Варианты переходов и предпочтения. Переход между одной и той же парой характеристик может иметь несколько вариантов, различающихся метками (`label`) и/или ограничениями. Поле `label_preference` в реестре задаёт порядок предпочтения, а стратегия вычислений может дополнительно выбирать вариант на основании `**options`.

5.3.3. Модуль families

Компонент **families** обеспечивает описание параметрических семейств распределений и построение конкретных экземпляров распределений из параметров. В отличие от **distributions**, который фиксирует «как вычислять», модуль **families** фиксирует «что такое распределение данного семейства»: набор параметров, допустимость значений, носитель и базовые (аналитические) характеристики.

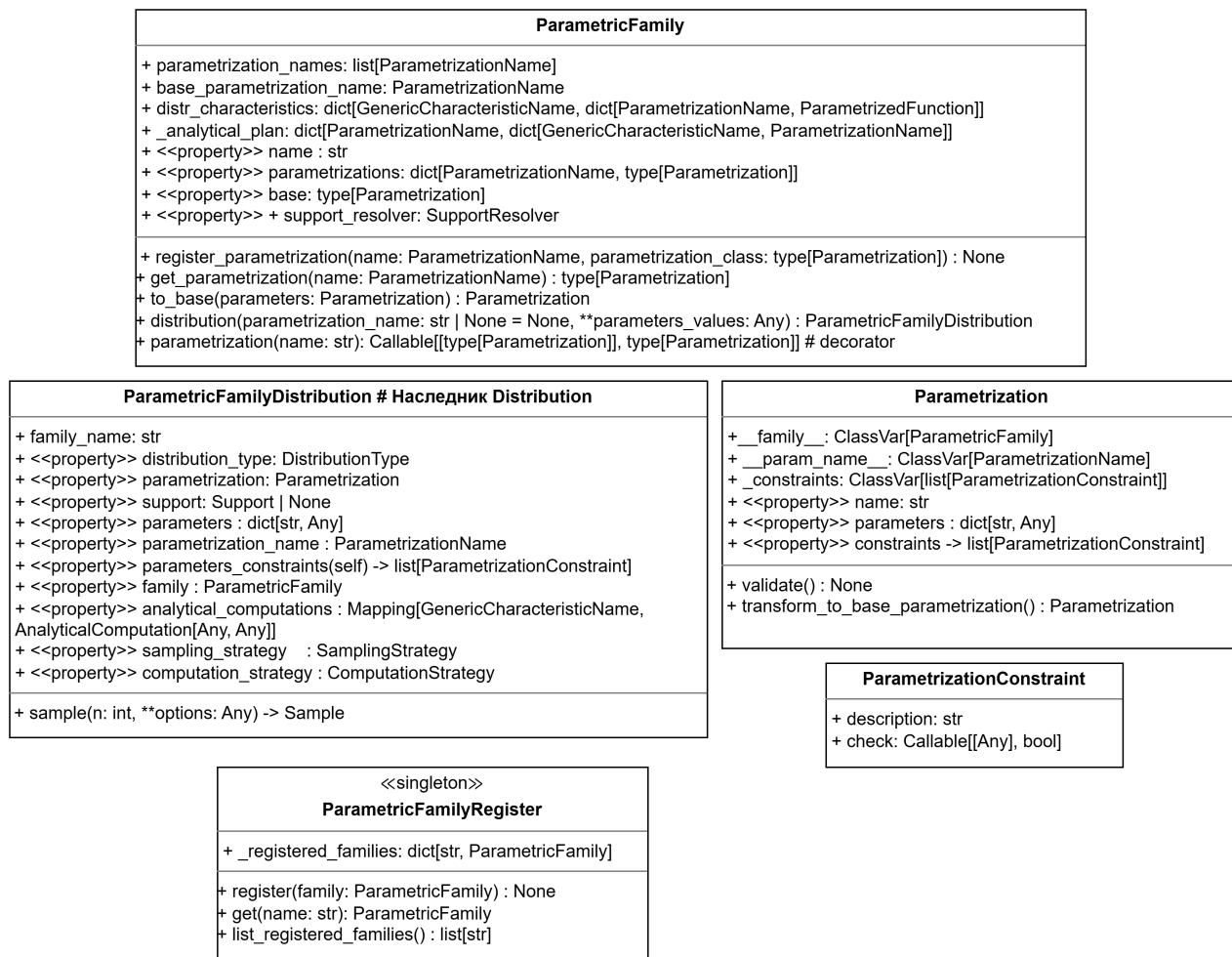


Рис. 5.7: Ключевые классы модуля families: семейство, параметризации, распределение и реестр семейств

5.3.3.1. Семейство распределений: ParametricFamily

ParametricFamily представляет одно семейство распределений и хранит:

- набор доступных параметризаций и имя базовой параметризации;
- правила приведения параметров к базовой форме;
- аналитические реализации характеристик, привязанные к параметризациям;
- **support_resolver** — функцию/объект, выводящий носитель распределения по параметрам.

Семейство предоставляет фабричный метод **distribution(...)**. Он принимает значения параметров (и, опционально, имя параметризации), валидирует их и возвращает объект **ParametricFamilyDistribution**, реализующий протокол **Distribution**. Таким образом, дальнейшие вычисления выполняются уже в инфраструктуре **distributions**.

5.3.3.2. Параметризации и ограничения

Параметризация представлена классом **Parametrization**. Экземпляр параметризации содержит словарь значений параметров и предоставляет:

- **validate()** — проверку набора ограничений;
- **transform_to_base_parametrization()** — преобразование параметров к базовой параметризации семейства.

Ограничения оформляются объектами **ParametrizationConstraint** (описание + предикат **check**). Важное свойство такой схемы состоит в том, что ограничения задаются рядом с параметризацией и остаются локальными: добавление новой параметризации не требует модификации существующих.

5.3.3.3. Аналитические характеристики и план вывода

Семейство хранит набор аналитических реализаций характеристик (на уровне `GenericCharacteristicName`) и допускает, что разные характеристики удобнее задавать в разных параметризациях. Для этого используется `analytical_plan`: он задаёт, к какой параметризации следует привести параметры, чтобы использовать доступную аналитическую формулу.

На уровне экземпляра распределения (`ParametricFamilyDistribution`) формируется отображение `analytical_computations`. Эти вычисления становятся «якорями» для дальнейшего вывода характеристик по реестру (см. §5.3.2.4).

5.3.3.4. Экземпляр распределения: `ParametricFamilyDistribution`

`ParametricFamilyDistribution` связывает воедино:

- семейство и выбранную параметризацию;
- конкретные значения параметров (в том числе в базовой форме);
- `support` и `distribution_type`;
- стратегии вычислений и сэмплирования.

Такое распределение является полноценным объектом `Distribution`: оно может генерировать выборку через `sampling_strategy` и вычислять характеристики через `computation_strategy`.

5.3.3.5. Глобальный реестр семейств

Для поддержки готовых семейств и интеграции с другими пакетами экосистемы введён синглтон `ParametricFamilyRegister`. Он предоставляет операции регистрации семейства, получения по имени и перечисления зарегистрированных семейств. Наличие отдельного реестра позволяет подключать «поставки» семейств из разных модулей, не связывая их напрямую.

5.4. Взаимодействия и потоки выполнения

В этом разделе фиксируются ключевые сценарии выполнения в вычислительной модели: поиск и применение метода вычисления характеристики, генерация выборки, а также точки расширения (регистрация новых правил и подмена стратегий).

5.4.1. Поток вычисления характеристики

Запрос характеристики начинается с обращения к объекту распределения через `query_method` или `calculate_characteristic`. Дальнейшее поведение делегируется стратегии вычислений:

1. **Аналитический якорь.** Если в `analytical_computations` присутствует реализация для целевой характеристики, стратегия возвращает `AnalyticalComputation`.
 2. **Кэш подогнанных методов.** При включённом кэшировании выполняется поиск уже построенного `FittedComputationMethod` для данной характеристики (и, при необходимости, набора `**options`). Если метод найден, он возвращается немедленно.
 3. **Построение по реестру.** Если аналитической реализации нет, стратегия запрашивает у глобального `CharacteristicRegistry` локальный вид графа: `view = registry.view(distr)`. В `RegistryView` выполняется поиск пути от доступных якорей (как правило, от характеристик, присутствующих в `analytical_computations`) к целевой характеристике. Поиск пути (`find_path`) выполняется с учётом:
 - наличия узлов и рёбер в локальном виде графа;
 - ограничений применимости (`constraints`), зависящих от признаков типа распределения и/или параметров экземпляра;
 - предпочтений по метке варианта (`prefer_label`) и глобального порядка `label_preference`.
 4. **Подгонка и композиция.** Для каждого шага найденного пути выбирается конкретный вариант перехода (`EdgeMeta`), после чего соответствующий `ComputationMethod` подгоняется под распределение через `fit(distr, **options)`. Полученная последовательность `FittedComputationMethod` компонуется в единый вызываемый объект для целевой характеристики.
 5. **Мемоизация.** Итоговый подогнанный метод сохраняется в кэш стратегии и возвращается вызывающей стороне.
- После получения метода `calculate_characteristic` применяет его к входному аргументу (например, к точке x для `pdf/cdf` или к массиву p для `ppf`).

5.4.2. Опции вычислений (`**options`)

Параметр `**options` используется как сквозной канал настройки вычислений и может влиять на:

- выбор варианта перехода в реестре (например, предпочтение численного/аналитического метода);
- поведение подгонки (`fit`) и подготовку внутренних структур;
- поведение вызова подогнанного метода (например, точность, ограничения итераций и т.п.).

Архитектурно `**options` трактуется как часть контекста вычисления: при наличии кэширования рекомендуется включать его (или его нормализованное представление) в ключ кэша.

5.4.3. Поток генерации выборки

Генерация выборки делегируется стратегии сэмплирования:

1. вызывается `distr.sample(n, **options)`;
2. внутри `sample` выполняется `sampling_strategy.sample(n, distr, **options)`;
3. результат возвращается в виде объекта, реализующего протокол `Sample` (обычно `ArraySample`).

Такое разделение позволяет:

- подключать специальные алгоритмы сэмплирования для отдельных распределений;
- переиспользовать представление выборки и последующую обработку в прикладных пакетах;
- тестировать стратегии независимо от конкретных семейств.

5.4.4. Точки расширения

Архитектура допускает расширение без модификации существующих интерфейсов:

- **Новые правила вывода.** Пользователь может добавить в `CharacteristicRegistry` новую характеристику или новое правило перехода (`add_characteristic`, `add_computation`), снабдив его ограничениями применимости.
- **Новые семейства и параметризации.** Через `ParametricFamilyRegister` (см. рис. 5.7) регистрируются новые семейства; в рамках семейства добавляются параметризации и ограничения.
- **Подмена стратегий.** Для распределения можно заменить `ComputationStrategy/SamplingStrategy`, что меняет политику выбора методов, кэширование и способ генерации выборки.

- **Варианты методов.** Для одной пары характеристик могут сосуществовать альтернативные реализации, различающиеся меткой и условиями применимости; стратегия выбирает вариант, используя предпочтения и ****options**.

Список литературы

- [1] Shun-ichi Amari. *Information geometry and its applications*. Т. 194. Springer, 2016.
- [2] Irina A Antipova. «Inversion of multidimensional Mellin transforms». В: *Russian Mathematical Surveys* 62.5 (2007), с. 977.
- [3] Ole Barndorff-Nielsen. *Information and exponential families: in statistical theory*. John Wiley & Sons, 2014.
- [4] Ole E Barndorff-Nielsen, Fred Espen Benth, Almut ED Veraart и др. *Ambit stochasticity*. Т. 88. Springer, 2018.
- [5] Richard Breen. *Regression models: Censored, sample selected, or truncated data*. 111. Sage, 1996.
- [6] Satish Chandra. «On the Mixtures of Probability Distributions». В: *Scandinavian Journal of Statistics* 4.3 (1977), с. 105–112.
- [7] David L Donoho и др. «High-dimensional data analysis: The curses and blessings of dimensionality». В: *AMS math challenges lecture 1.2000* (2000), с. 32.
- [8] William Palin Elderton и Norman Lloyd Johnson. «Systems of frequency curves». В: *(No Title)* (1969).
- [9] Felix Famoye. *Continuous univariate distributions, volume 1*. 1995.
- [10] Alessio Figalli. «On the continuity of center-outward distribution and quantile functions». В: *Nonlinear Analysis* 177 (2018), с. 413–421.
- [11] Janos Galambos и Italo Simonelli. *Products of random variables: applications to problems of physics and to arithmetical functions*. CRC press, 2004.
- [12] Marc Hallin и Dimitri Konen. «Multivariate Quantiles: Geometric and Measure-Transportation-Based Contours». В: *Applications of Optimal Transport to Economics and Related Topics*. Springer, 2024, с. 61–78.
- [13] Daniel Hernandez-Stumpfhauser, F. Jay Breidt и Mark J. van der Woerd. «The General Projected Normal Distribution of Arbitrary Dimension: Modeling and Bayesian Inference». В: *Bayesian Analysis* 12.1 (2017), с. 113–133.
- [14] Wolfgang Hörmann, Josef Leydold и Gerhard Derflinger. *Automatic nonuniform random variate generation*. Springer Science & Business Media, 2013.
- [15] JRM Hosking. «L-moments». В: *Wiley StatsRef: Statistics Reference Online* (2014), с. 1–8.
- [16] Thomas W Keelin. «The metalog distributions». В: *Decision Analysis* 13.4 (2016), с. 243–277.
- [17] David G Kleinbaum и Mitchel Klein. *Survival analysis a self-learning text*. Springer, 1996.
- [18] Donald E Knuth. *The Art of Computer Programming: Seminumerical Algorithms, Volume 2*. Addison-Wesley Professional, 2014.
- [19] Solomon Kullback. *Information theory and statistics*. Courier Corporation, 1997.
- [20] Arnaud de La Fortelle. «A study on generalized inverses and increasing functions Part I: generalized inverses». working paper or preprint. Август. 2015. URL: <https://minesparis-psl.hal.science/hal-01255512>.
- [21] Nicolas Lanchier. *Stochastic modeling*. Springer, 2017.
- [22] Lawrence M Leemis и др. «Univariate probability distributions». В: *Computational Probability Applications* (2017), с. 133–147.
- [23] Erich L Lehmann и George Casella. *Theory of point estimation*. Springer Science & Business Media, 2006.
- [24] Erich Leo Lehmann и др. «Statistical methods based on ranks». В: *Nonparametrics. San Francisco, CA, Holden-Day* 2 (1975).
- [25] Faming Liang, Chuanhai Liu и Raymond Carroll. *Advanced Markov chain Monte Carlo methods: learning from past samples*. John Wiley & Sons, 2011.
- [26] Govind S Mudholkar и Alan D Hutson. «LQ-moments: Analogs of L-moments». В: *Journal of Statistical Planning and Inference* 71.1-2 (1998), с. 191–208.

- [27] Frank Nielsen и Vincent Garcia. «Statistical exponential families: A digest with flash cards». В: *arXiv preprint arXiv:0911.4863* (2009).
- [28] Luigi Pace и Alessandra Salvan. *Principles of statistical inference: from a Neo-Fisherian perspective*. Т. 4. World scientific, 1997.
- [29] Xavier Pennec. «Probabilities and statistics on Riemannian manifolds: Basic tools for geometric measurements.» В: *NSIP*. Т. 3. 1999, с. 194—198.
- [30] Karsten Prause и др. «The generalized hyperbolic model: Estimation, financial derivatives, and risk measures». Дис. ... док. Citeseer, 1999.
- [31] David Ruppert. «What is kurtosis? An influence function approach». В: *The American Statistician* 41.1 (1987), с. 1—5.
- [32] Jacob Schreiber. «Pomegranate: fast and flexible probabilistic modeling in python». В: *Journal of Machine Learning Research* 18.164 (2018), с. 1—6.
- [33] Melvin Dale Springer. *The algebra of random variables*. New York: Wiley, 1979.
- [34] György Steinbrecher и William T Shaw. «Quantile mechanics». В: *European journal of applied mathematics* 19.2 (2008), с. 87—112.
- [35] Maciej J Swat, Pierre Grenon и Sarala Wimalaratne. «ProbOnto: ontology and knowledge base of probability distributions». В: *Bioinformatics* 32.17 (2016), с. 2719—2721.
- [36] Herbert Weisberg. *Central tendency and variability*. 83. Sage, 1992.
- [37] Вильям Феллер. *Введение в теорию вероятностей и ее приложения*. Рипол Классик, 2013.
- [38] Наталья Исааковна Чернова. *Математическая статистика*. Новосибирский гос. ун-т, 2007.
- [39] Альберт Николаевич Ширяев. *Вероятность*. МЦНМО, 2007.